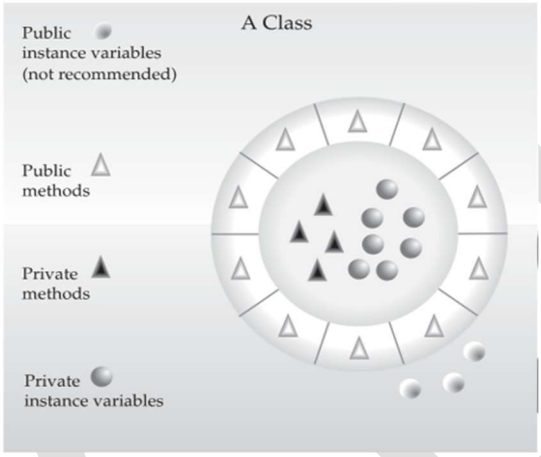
Module 1 - An Overview of Java

1. Explain The Three OOP Principles

Encapsulation

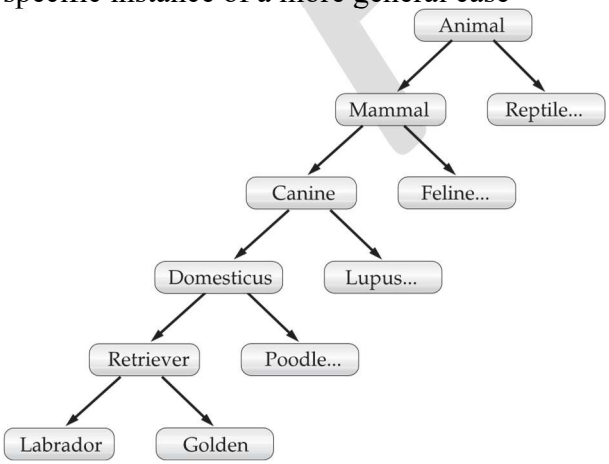
Encapsulation is the mechanism that binds together code and the data it manipulates, and keeps both safe from outside interference and misuse. One way to think about encapsulation is as a protective wrapper that prevents the code and data from being arbitrarily accessed by other code defined outside the wrapper. Access to the code and data inside the wrapper is tightly controlled through a well-defined interface. , there are mechanisms for hiding the complexity of the implementation inside the class. Each method or variable in a class may be marked private or public.

FIGURE 2-1
Encapsulation: public methods can be used to protect private data



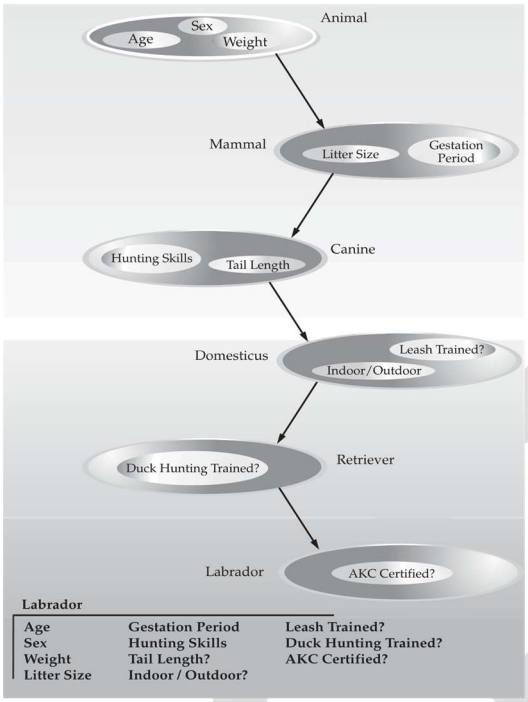
Inheritance

Inheritance is the process by which one object acquires the properties of another object. This is important because it supports the concept of hierarchical classification. As mentioned earlier, most knowledge is made manageable by hierarchical (that is, top-down) classifications. For example, a Golden Retriever is part of the classification *dog,* which in turn is part of the *mammal* class, which is under the larger class *animal.* Without the use of hierarchies, each object would need to define all of its characteristics explicitly. However, by use of inheritance, an object need only define those qualities that make it unique within its class. It can inherit its general attributes from its parent. Thus, it is the inheritance mechanism that makes it possible for one object to be a specific instance of a more general case



Polymorphism

Polymorphism (from Greek, meaning "many forms") is a feature that allows one interface to be used for a general class of actions. The specific action is determined by the exact nature of the situation.



---

2. Explain Two Control Statements with example

The if Statement

The Java if statement works much like the IF statement in any other language. Further, it is syntactically identical to the if statements in C, C++, and C#. Its simplest form is shown here:

if(*condition*) *statement*;

Here, *condition* is a Boolean expression. If *condition* is true, then the statement is executed. If *condition* is false, then the statement is bypassed. Here is an example:

```
if(num < 100)
System.out.println("num is less than 100");
```

Example

```
class IfSample {
public static void main(String args[]) {
int x, y;
x = 10; y = 20;
if(x < y) System.out.println("x is less than y");
x = x * 2; if(x == y) System.out.println("x now equal to y");
x = x * 2;
if(x > y) System.out.println("x now greater than y");
// this won't display anything if(x == y) System.out.println("you won't see this");
}
}
```

The output generated by this program is shown here:
x is less than y x now equal to y x now greater than y

The for Loop

for(*initialization; condition; iteration*)
statement;

In its most common form, the *initialization* portion of the loop sets a loop control variable to an initial value. The *condition* is a Boolean expression that tests the loop control variable. If the outcome of that test is true, the **for** loop continues to iterate. If it is false, the loop terminates. The *iteration* expression determines how the loop control variable is changed each time the loop iterates. Here is a short program that illustrates the **for** loop:

```
class ForTest {
public static void main(String args[]) {
int x;
for(x = 0; x<10; x = x+1)
System.out.println("This is x: " + x);
}
}
```

This program generates the following output:

```
This is x: 0
This is x: 1
This is x: 2
This is x: 3
This is x: 4
This is x: 5
This is x: 6
This is x: 7
This is x: 8
This is x: 9
```

3. Explain object oriented principles.

Encapsulation

Encapsulation is the mechanism that binds together code and the data it manipulates, and keeps both safe from outside interference and misuse. One way to think about encapsulation is as a protective wrapper that prevents the code and data from being arbitrarily accessed by other code defined outside the wrapper. Access to the code and data inside the wrapper is tightly controlled through a well-defined interface. , there are mechanisms for hiding the complexity of the implementation inside the class. Each method or variable in a class may be marked private or public.

ACET

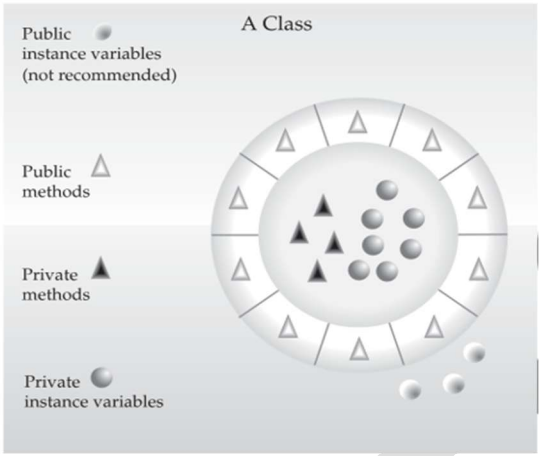
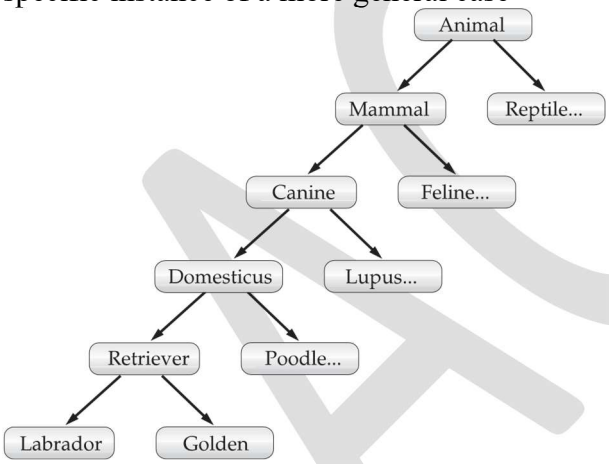


FIGURE 2-1
Encapsulation: public methods can be used to protect private data

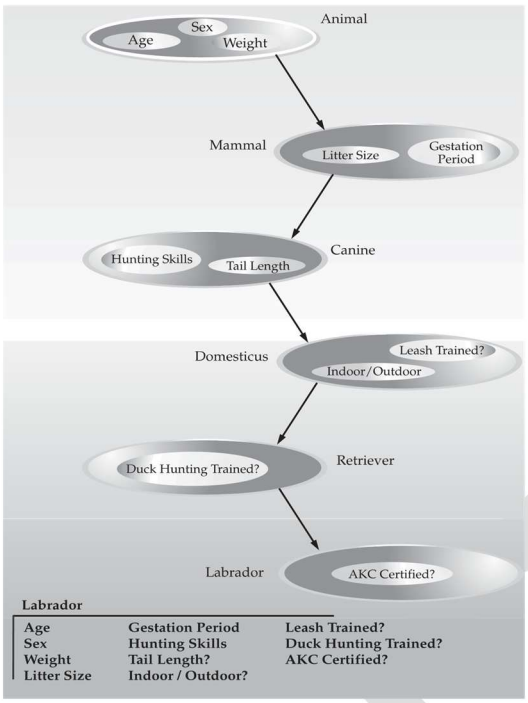
Inheritance

Inheritance is the process by which one object acquires the properties of another object. This is important because it supports the concept of hierarchical classification. As mentioned earlier, most knowledge is made manageable by hierarchical (that is, top-down) classifications. For example, a Golden Retriever is part of the classification *dog,* which in turn is part of the *mammal* class, which is under the larger class *animal.* Without the use of hierarchies, each object would need to define all of its characteristics explicitly. However, by use of inheritance, an object need only define those qualities that make it unique within its class. It can inherit its general attributes from its parent. Thus, it is the inheritance mechanism that makes it possible for one object to be a specific instance of a more general case



Polymorphism

Polymorphism (from Greek, meaning "many forms") is a feature that allows one interface to be used for a general class of actions. The specific action is determined by the exact nature of the situation.



Abstraction

An essential element of object-oriented programming is *abstraction.* Humans manage complexity through abstraction. For example, people do not think of a car as a set of tens of thousands of individual parts. They think of it as a well-defined object with its own unique behavior.

In Java, **abstraction** is a key concept in object-oriented programming that lets you focus on *what* an object does rather than *how* it does it. It helps simplify complex systems by hiding unnecessary details and showing only the essential features.

4. Explain different lexical issues in JAVA.

Whitespace

Java is a free-form language. This means that you do not need to follow any special indentation rules. For instance, the Example program could have been written all on one line or in any other strange way you felt like typing it, as long as there was at least one whitespace character between each token that was not already delineated by an operator or separator. In Java, whitespace is a space, tab, or newline.

Identifiers

Identifiers are used for class names, method names, and variable names. An identifier may be any descriptive sequence of uppercase and lowercase letters, numbers, or the underscore and dollar-sign characters. They must not begin with a number, lest they be confused with a numeric literal. Again, Java is case-sensitive, so VALUE is a different identifier than Value.

Some examples of valid identifiers are

AvgTemp	**count**	**a4**	**$test**	**this_is_ok**

Invalid identifier names include these:

2count	**high-temp**	**Not/ok**

Literals

A constant value in Java is created by using a *literal* representation of it. For example, here are some literals:

Left to right, the first literal specifies an integer, the next is a floating-point value, the third is a character constant, and the last is a string. Aliteral can be used anywhere a value of its type is allowed.

100	**98.6**	**'X'**	**"This is a test"**

Comments

there are three types of comments defined by Java. single-line and multiline. The third type is called a *documentation comment.* This type of comment is used to produce an HTMLfile that documents your program. The documentation comment begins with a /** and ends with a */.

Separators

In Java, there are a few characters that are used as separators. The most commonly used separator in Java is the semicolon. As you have seen, it is used to terminate statements. The separators are shown in the following table:

Symbol	Name	Purpose
( )	**Parentheses**	**Used to contain lists of parameters in method definition and invocation. Also used for defining precedence in expressions, containing expressions in control statements, and surrounding cast types.**
{ }	**Braces**	**Used to contain the values of automatically initialized arrays. Also used to define a block of code, for classes, methods, and local scopes.**
[ ]	**Brackets**	**Used to declare array types. Also used when dereferencing array values.**
;	**Semicolon**	**Terminates statements.**
,	**Comma**	**Separates consecutive identifiers in a variable declaration. Also used to chain statements together inside a for statement.**
.	**Period**	**Used to separate package names from subpackages and classes. Also used to separate a variable or method from a reference variable.**

The Java Keywords

There are 50 keywords currently defined in the Java language. These keywords, combined with the syntax of the operators and separators, form the foundation of the Java language. These keywords cannot be used as names for a variable, class, or method.

abstract	**continue**	**for**	**new**	**switch**
assert	**default**	**goto**	**package**	**synchronized**

boolean	**do**	**if**	**private**	**this**
break	**double**	**implements**	**protected**	**throw**
byte	**else**	**import**	**public**	**throws**
case	**enum**	**instanceof**	**return**	**transient**
catch	**extends**	**int**	**short**	**try**
char	**final**	**interface**	**static**	**void**
class	**finally**	**long**	**strictfp**	**volatile**
const	**float**	**native**	**super**	**while**

5. Explain Block of code with an example

Java allows two or more statements to be grouped into *blocks of code,* also called *code blocks.* This is done by enclosing the statements between opening and closing curly braces. Once a block of code has been created, it becomes a logical unit that can be used any place that a single statement can.

For example, a block can be a target for Java's if and for statements. Consider this if statement:

```
if(x < y) { // begin a block
x = y; y = 0;
} // end of block

class BlockTest {
public static void main(String args[]) {
int x, y;
y = 20;
for(x = 0; x<10; x++) {
System.out.println("This is x: " + x);
System.out.println("This is y: " + y);
y = y - 2;
}
}
}
```

The output generated by this program is shown here:

```
This is x: 0
This is y: 20
This is x: 1
This is y: 18
This is x: 2
This is y: 16
This is x: 3
This is y: 14
This is x: 4
This is y: 12
```

ACET

```
This is x: 5
This is y: 10
This is x: 6
This is y: 8
This is x: 7
This is y: 6
This is x: 8
This is y: 4
This is x: 9
This is y: 2
```

6. Narrate The Primitive data Types in Java

Primitive data types are predefined by the Java language and represent raw values like numbers, characters, and logical states. They are not associated with any methods or behaviors and offer fast and efficient data operations.

Key Characteristics

Efficient: Use less memory and offer quicker performance than objects.

Immutable: Once assigned a value, it cannot be changed directly.

Default Values: Automatically initialized in arrays or class fields.

The 8 Primitive Data Types in Java

Type	Purpose	Example
byte	Very small integer	byte b = 100;
short	Small integer	short s = 30000;
int	Standard integer	int i = 123456;
long	Large integer	long l = 123456789L;
float	Single-precision decimal	float f = 3.14f;
double	Double-precision decimal	double d = 2.71828;
char	Single character (Unicode)	char c = 'A';

7. Explain different type of integer literals with examples.

I**nteger literals** represent fixed values written directly in code for integral types like byte, short, int, and long. Java provides several formats for writing these literals, which can be especially useful for readability or specific use cases.

```
public class IntegerLiteralsExample {
   public static void main(String[] args) {
      int decimal = 123;          // Decimal literal
      int binary = 0b1111011;     // Binary literal (equals 123)
      int octal = 0173;           // Octal literal (equals 123)
      int hex = 0x7B;             // Hexadecimal literal (equals 123)
      int readable = 1_000_000;   // Using underscores for clarity
```

```
        System.out.println("Decimal: " + decimal);
        System.out.println("Binary: " + binary);
        System.out.println("Octal: " + octal);
        System.out.println("Hexadecimal: " + hex);
        System.out.println("Readable Format: " + readable);
    }
}
```


Decimal: 123
Binary: 123
Octal: 123
Hexadecimal: 123
Readable Format: 1000000

8. Write a program to multiply with two arrays (one dimonsional)

```
public class ArrayMultiplication {
    public static void main(String[] args) {
        int[] array1 = {2, 4, 6, 8, 10};
        int[] array2 = {1, 3, 5, 7, 9};
        int[] result = new int[array1.length];

        for (int i = 0; i < array1.length; i++) {
            result[i] = array1[i] * array2[i];
        }

        System.out.println("Result of multiplying two arrays:");
        for (int val : result) {
            System.out.print(val + " ");
        }
    }
}
```

Output
Result of multiplying two arrays:
2 12 30 56 90

9. Explain Scope and Lifetime of Variables.

the variables used have been declared at the start of the **main( )** method. However, Java allows variables to be declared within any block. a block is begun with an opening curly brace and ended by a closing curly brace. A block defines a *scope.* Thus, each time you start a new block, you are creating a new scope. A scope determines what objects are visible to other parts of your program. It also determines the lifetime of those objects.

Many other computer languages define two general categories of scopes: global and local. However, these traditional scopes do not fit well with Java's strict, object-oriented model.

```
class Scope {
public static void main(String args[]) {
int x;
```

```
x = 10; if(x == 10) {
int y = 20;
System.out.println("x and y: " + x + " " + y);
x = y * 2;
}
 y = 100;
System.out.println("x is " + x);
}
}
```

Example for lifetime

```
class LifeTime {
public static void main(String args[]) {
int x;
for(x = 0; x < 3; x++) {
int y = -1;
System.out.println("y is: " + y);
y = 100;
System.out.println("y is now: " + y);
}
}
}
```

The output generated by this program is shown here:

```
y is: -1
y is now: 100
y is: -1
y is now: 100
y is: -1
y is now: 100
```

10. Explain different promotion rules in JAVA.

Java defines several *type promotion* rules that apply to expressions. They are as follows: First, all **byte**, **short**, and **char** values are promoted to **int**, as just described. Then, if one operand is a **long**, the whole expression is promoted to **long**. If one operand is a **float,** the entire expression is promoted to **float**. If any of the operands is **double**, the result is **double**.

The following program demonstrates how each value in the expression gets promoted to match the second argument to each binary operator:

```
class Promote {
public static void main(String args[]) {
byte b = 42;
char c = 'a';
short s = 1024;
int i = 50000;
```

ACET

```
float f = 5.67f;
double d = .1234;
double result = (f * b) + (i / c) - (d * s);
System.out.println((f * b) + " + " + (i / c) + " - " + (d * s));
System.out.println("result = " + result);
}
}
```

double result = (f * b) + (i / c) - (d * s);

11. Explain Arrays in java.

An *array* is a group of like-typed variables that are referred to by a common name. Arrays of any type can be created and may have one or more dimensions. Aspecific element in an array is accessed by its index. Arrays offer a convenient means of grouping related information.

One-Dimensional Arrays

A *one-dimensional array* is, essentially, a list of like-typed variables. To create an array, you first must create an array variable of the desired type. The general form of a one-dimensional array declaration is

type var-name[ ];

Here, *type* declares the base type of the array.

array-var = new *type*[*size*];

Example

```
class Array {
public static void main(String args[]) {
int month_days[];
month_days = new int[12];
month_days[0] = 31;
month_days[1] = 28;
month_days[2] = 31;
month_days[3] = 30;
month_days[4] = 31;
month_days[5] = 30;
month_days[6] = 31;
month_days[7] = 31;
month_days[8] = 30;
month_days[9] = 31;
month_days[10] = 30;
month_days[11] = 31;
System.out.println("April has " + month_days[3] + " days.");
}
}
```

Multidimensional Arrays

ACET

In Java, *multidimensional arrays* are actually arrays of arrays. These, as you might expect, look and act like regular multidimensional arrays. To declare a multidimensional array variable, specify each additional index using another set of square brackets.
For example, the following declares a twodimensional array variable called **twoD**. int twoD[][] = new int[4][5];
This allocates a 4 by 5 array and assigns it to **twoD**. Internally this matrix is implemented as an *array* of *arrays* of **int**.

```
class TwoDArray {
public static void main(String args[]) {
int twoD[][]= new int[4][5]; int i, j, k = 0;
for(i=0; i<4; i++) for(j=0; j<5; j++) {
twoD[i][j] = k; k++;
}
for(i=0; i<4; i++) { for(j=0; j<5; j++)
System.out.print(twoD[i][j] + " ");
System.out.println();
}
}
}
```

This program generates the following output:

```
0 1 2 3 4
5 6 7 8 9
10 11 12 13 14
15 16 17 18 19
```

12. Explain the structure and components of a Java compilation unit with suitable illustration.

```
class Example {
public static void main(String args[]) {
System.out.println("This is a simple Java program.");
}
}
```

Compiling the Program
To compile the **Example** program, execute the compiler, **javac**, specifying the name of the source file on the command line, as shown here:

```
C:\>javac Example.java
```

The **javac** compiler creates a file called **Example.class** that contains the bytecode version of the program. As discussed earlier, the Java bytecode is the intermediate representation of your program that contains instructions the Java Virtual Machine will execute. Thus, the output of **javac** is not code that can be directly executed.

To actually run the program, you must use the Java application launcher, called **java**.
To do so, pass the class name **Example** as a command-line argument, as shown here:
C:\>java Example

When the program is run, the following output is displayed:
This is a simple Java program.

class Example {
This line uses the keyword class to declare that a new class is being defined. Example is an *identifier* that is the name of the class. The entire class definition, including all of its members, will be between the opening curly brace ({) and the closing curly brace (}).
Public static void main(String args[]) {
All Java applications begin execution by calling **main( )**.
public

- Access modifier that lets other classes call the method.
- Needed for main() so the JVM can invoke it from outside its class.

static

- Means you don't need to create an object to call the method.
- Required for main() because it's run before any class instances exist.

void

- Indicates the method does not return a value.
- main() performs tasks but doesn't return anything to the caller.

String[] args:

- Command-line arguments passed at runtime.
- Acts like an inbox for input when launching the program.

System.out.println("This is a simple Java program.");

This line outputs the string "This is a simple Java program." followed by a new line on the screen. Output is actually accomplished by the built-in println( ) method